

Assignment 4 solution

1. 【考点：传输层功能与寻址】某 Web 服务器（IP 地址为 192.168.1.100）同时运行了 HTTP 网站服务（监听 TCP 80 端口）和 DNS 解析服务（监听 UDP 53 端口）。此时有三台客户机同时与该服务器进行通信：主机 A（IP 10.0.0.1）使用 TCP 50000 端口与该服务器建立了一个 HTTP 连接；主机 B（IP 10.0.0.2）也使用 TCP 50000 端口与该服务器建立了一个 HTTP 连接；主机 C（IP 10.0.0.3）使用 UDP 50000 端口向该服务器发送了一个 DNS 查询。关于服务器端传输层的复用与分用机制，下列说法正确的是（ ）

- A. 服务器端传输层收到主机 A 和主机 B 的 TCP 报文段后，由于目的端口和源端口均相同，无法在传输层进行区分，必须依靠应用层的 HTTP 报文首部进行分离
- B. 服务器端操作系统中，为这三个并发的通信过程总共创建了 2 个 Socket（套接字）用于接收数据
- C. 服务器侧的 TCP 模块依靠“源 IP 地址、源端口号、目的 IP 地址、目的端口号”这个四元组来唯一标识和区分主机 A 与主机 B 的连接
- D. UDP 属于无连接协议，因此服务器侧的 UDP 模块只需依靠“目的端口号”就能将主机 C 的查询准确交付给 DNS 进程，无需检查源 IP 和源端口

【答案】：C

【解析】：

- A 错、C 正确：TCP 是面向连接的，TCP 连接由一个四元组（源 IP、源端口、目的 IP、目的端口）唯一确定。虽然 A 和 B 的目的 IP、目的端口、甚至源端口（50000）都相同，但它们的源 IP 地址不同。因此服务器的 TCP 模块能通过四元组准确分用，不需要依赖应用层
- B 错：在服务器端，TCP 监听端口（80）会占用 1 个 Socket；当 A 和 B 分别连接成功后，服务器会为每个并发连接各生成 1 个新的已连接 Socket；UDP 服务也会占用 1 个 Socket。所以至少创建了 $1 + 2 + 1 = 4$ 个 Socket
- D 错：UDP 虽然是无连接的，依靠目的端口号将报文交付给对应的应用进程，但在进行应用层回复时，必须提取并使用源 IP 和源端口。因此 UDP socket 接收数据时同样会向上层提供源 IP 和源端口信息，并非“无需检查”

2. 【考点：UDP 校验与伪首部】在计算 UDP 校验和时，需要在 UDP 数据报前临时添加一个伪首部。关于 UDP 校验和以及伪首部的作用，下列说法错误的是（ ）

- A. 伪首部中包含了源 IP 地址和目的 IP 地址，其目的是让 UDP 传输层能够双重检查该数据报是否真的应该投递给本机的该目的端口
- B. 若 UDP 数据报的数据部分长度为奇数个字节，在计算校验和时会在数据部分末尾临时填充一个全 0 字节，但该填充字节不会被发送到网络中
- C. UDP 的校验和字段是可选的。若发送端不想计算校验和，可以将该字段全填为 0；若计算出的校验和刚好为全 0，则将该字段全填为 1
- D. 当 UDP 报文穿过 NAT（网络地址转换）路由器时，由于伪首部仅在端系统内部临时生成参与计算，NAT 路由器只需修改 IP 首部，无需重新计算 UDP 的校验和

【答案】：D

【解析】：

- A 正确：伪首部包含 IP 层信息，可以防止 IP 路由错误导致数据报被送错主机
- B 正确：UDP 校验和按 16 位（2 字节）字进行计算，奇数长度需临时补零计算，但不发送补零字节
- C 正确：UDP 校验和值为 0 有特殊含义（未计算校验和），如果真算出来是 0，标准规定填入全 1（反码算术中的负零）
- D 错误：NAT 路由器改变了 IP 数据报的源 IP 或目的 IP。因为伪首部包含了 IP 地址，IP 地址改变会导致原本计算出的 UDP 校验和失效。因此，NAT 路由器除了修改 IP 头，还必须深入传输层，重新计算并修改 UDP 的校验和字段

3. 【考点：TCP 段结构与序号耗用】甲、乙两台主机建立了 TCP 连接，甲向乙发送了 3 个连续的 TCP 数据段，其序号（Sequence Number）分别是 70、100、150。已知第三个数据段携带了 30 字节的有效载荷。假设不考虑任何 TCP 选项，且这些数据段都只带有正常的 ACK 确认（不带 SYN 或 FIN 标志）。下列关于这几个 TCP 段的推断中，完全正确的是（ ）

- A. 第一个数据段包含 30 字节的有效载荷
- B. 第二个数据段包含 51 字节的有效载荷
- C. 乙主机在成功接收这三个数据段后，向甲发送的确认号（Acknowledgment Number）应为 179
- D. 若这三个数据段的首部长度字段（数据偏移）的值为 0101（二进制），则表明它们都携带有 20 字节的 TCP 选项

【答案】：A

【解析】：

TCP 序号是面向字节流的，每个字节对应一个序号。下一个段的序号 = 上一个段的序号 + 上一个段的数据长度

A 正确：段 2 序号为 100，段 1 序号为 70。段 1 长度 = $100 - 70 = 30$ 字节

B 错：段 3 序号为 150，段 2 序号为 100。段 2 长度 = $150 - 100 = 50$ 字节，并非 51 字节

C 错：段 3 的序号为 150，长度为 30 字节。它包含的字节序号为 150~179。因此，乙期望收到的下一个字节序号是 180，确认号应为 180

D 错：数据偏移字段单位是 4 字节。二进制 0101 即 5，表示首部长度为 $5 \times 4 = 20$ 字节。这是 TCP 固定首部的最小长度，表明没有携带任何 TCP 选项

4. 【考点：TCP 连接管理】在 TCP 三次握手过程中，为了防御 SYN Flood（SYN 泛洪）攻击，现代操作系统广泛采用了 SYN Cookie 技术。关于传统的三次握手与 SYN Cookie 介入后的状态机对比，下列描述正确的是（ ）

- A. 在传统握手中，服务器收到客户端的 SYN 报文后，会立即分配接收缓存和发送缓存，并进入 SYN_RCVD 状态
- B. 启用 SYN Cookie 后，服务器收到 SYN 报文时不再为该半连接分配任何状态内存，而是利用算法计算出一个特殊的初始序号（Cookie）回复给客户端
- C. 无论是否启用 SYN Cookie，客户端发送第三次握手的 ACK 报文时，该报文都会消耗掉客户端的一个序列号
- D. 在正常的三次握手中，第二步的 SYN+ACK 报文如果丢失，服务器的重传计时器不会触发重传，只有客户端会重发 SYN

【答案】：B

【解析】：

A 错：传统握手中，服务器收到 SYN 后会分配一种叫“半连接队列”（TCB）的数据结构并进入 SYN_RCVD 状态，但通常不会立即分配接收缓存和发送缓存（缓存通常在第三次握手成功进入 ESTABLISHED 状态时才分配，以节约资源）

B 正确：这正是 SYN Cookie 的核心思想。通过丢弃状态，将连接信息编码在序号中，只有当合法的带有准确 ACK 号的第三次握手到来时，才重新还原状态，从根本上防止了内存耗尽

C 错：纯粹的 ACK 报文（不携带数据，且没有 SYN 或 FIN 标志）不消耗任何序列号

D 错：服务器处于 SYN_RCVD 状态时，如果迟迟收不到第三次握手的 ACK，服务器会触发超时重传，主动重发 SYN+ACK 报文

5. 【考点：TCP 连接管理】主机甲和主机乙之间建立了一条 TCP 连接。现主机甲主动发起断开连接的请求，发送了 FIN 报文。主机乙收到后回复了 ACK 报文，但此时乙仍有大量数据需要发送给甲。在乙发送完所有剩余数据，并最终向甲发送 FIN 报文之后，甲回复了最后的 ACK。下列关于此过程中双方状态及通信机制的描述，正确的是（ ）

- A. 甲发送第一个 FIN 报文并收到乙的 ACK 后，甲进入 TIME_WAIT 状态，此时甲不能再发送数据，也不能接收数据
- B. 乙在回复第一个 ACK 后，进入 CLOSE_WAIT 状态，此时这条 TCP 连接处于“半关闭”状态
- C. 甲在收到乙的 FIN 报文后，立即进入 CLOSED 状态并释放该连接占用的所有端口资源

D. 乙在发送完最后的 FIN 报文后，进入 LAST_ACK 状态，一旦收到甲的确认，乙也需要等待 2MSL 的时间才能进入 CLOSED 状态

【答案】：B

【解析】：

A 错：甲发 FIN 收 ACK 后，进入 FIN_WAIT_2 状态，此时甲不能再发数据，但完全可以接收乙发来的数据

B 正确：被动关闭方收到 FIN 并回复 ACK 后，进入 CLOSE_WAIT 状态。此时甲到乙的方向关闭，乙到甲的方向仍然敞开，称为“半关闭”

C 错：甲收到乙的 FIN 并回复 ACK 后，进入 TIME_WAIT 状态，必须等待 2MSL（最长报文段寿命的两倍）时间后，才能进入 CLOSED 状态并释放端口

D 错：只有主动关闭方（甲）需要进入 TIME_WAIT 状态并等待 2MSL。被动关闭方（乙）收到最后的 ACK 后，直接进入 CLOSED 状态

6. 【考点：TCP 可靠传输与 RTT 估算】TCP 协议采用自适应算法来动态调整超时重传时间（RTO）。假设一台主机的 Smoothed RTT (SRTT) 当前值为 30 ms，RTT 偏差 (RTTVAR) 当前值为 5 ms。此时该主机发送了一个 TCP 段，并测得其实际往返时间 (SampleRTT) 为 40 ms。已知标准算法中 $\alpha = 0.125$ (1/8)， $\beta = 0.25$ (1/4)。随后，该主机发送的下一个 TCP 段发生了超时重传。关于该过程中的 RTO 变化，下列计算及机制描述正确的是（）

A. 收到 40 ms 的采样后，新的 SRTT 更新为 35 ms

B. 收到 40 ms 的采样后，新的 RTO 被更新为 56 ms

C. 当下一个报文段发生超时重传时，为了防止网络持续拥塞，TCP 会通过 Karn 算法将当前的 RTO 值减半

D. 超时重传的报文段在收到确认后，TCP 会立即使用该报文段的往返时间采样值去更新 SRTT

【答案】：B

【解析】：

公式： $SRTT_{new} = (1 - \alpha) \times SRTT_{old} + \alpha \times SampleRTT$

公式： $RTTVAR_{new} = (1 - \beta) \times RTTVAR_{old} + \beta \times |SampleRTT - SRTT_{old}|$

公式： $RTO = SRTT_{new} + 4 \times RTTVAR_{new}$

A 错： $SRTT_{new} = (7/8) \times 30 + (1/8) \times 40 = 26.25 + 5 = 31.25$ ms

B 正确：先计算 $RTTVAR_{new} = (3/4) \times 5 + (1/4) \times |40 - 30| = 3.75 + 2.5 = 6.25$ ms。然后计算 $RTO = 31.25 + 4 \times 6.25 = 31.25 + 25 = 56.25$ ms ≈ 56 ms

C 错：发生超时重传时，TCP 会执行“指数退避”，即把 RTO 的值加倍（乘以 2），而不是减半

D 错：在计算 SRTT 时，只要发生重传，就不采用该重传报文段的 RTT 样本，因为无法确定确认是针对原始报文还是重传报文

7. 【考点：TCP 流量控制与零窗口探测】主机甲向主机乙连续发送大文件。在通信过程中，由于乙的应用层进程读取数据缓慢，乙向甲发送了一个带有 `rwnd = 0`（接收窗口为零）标志的 ACK 报文。为了打破死锁并恢复传输，TCP 协议设计了零窗口探测机制。下列关于此场景下流量控制的说法中，正确的是（）

A. 甲收到 `rwnd = 0` 后，会停止发送任何类型的数据报文，直到收到乙主动发来的窗口更新报文

B. 甲启动持续计时器，超时后会发送一个仅携带 1 字节数据的零窗口探测报文，该报文不消耗甲的序列号

C. 即使乙发送了零窗口通告，TCP 协议仍然允许甲强制向乙发送紧急数据（带 URG 标志）以通知乙的中断事件

D. TCP 流量控制主要依靠控制拥塞窗口 (cwnd) 的大小来实现，接收窗口 (rwnd) 仅起到辅助作用

【答案】：C

【解析】：

A 错：如果乙的窗口更新报文丢失，会死锁。甲启动持续计时器主动探测，而不是被动死等

B 错：零窗口探测报文通常携带 1 字节的有效数据，这 1 字节是会消耗序列号的。乙收到后，如果窗口仍为 0，会丢弃这 1 字节，并在回复的 ACK 中包含同样的确认号和零窗口通告

C 正确：这是 TCP 流量控制的例外情况。即使接收窗口为 0，发送端也被允许发送以下三种报文：零窗口探测报文、确认报文（ACK）、以及携带紧急数据（URG=1）的报文

D 错：流量控制是端到端的机制，主要依靠接收窗口（rwnd）来实现；而限制网络注入速率的拥塞窗口（cwnd）是拥塞控制的核心，两者结合共同决定发送窗口： $W = \min(rwnd, cwnd)$

8. 【考点：TCP 拥塞控制】某条 TCP 连接采用标准算法进行拥塞控制。连接初始化时，拥塞窗口 $cwnd = 1$ KB，慢开始门限 $ssthresh = 16$ KB。发送端开始发送数据，且假设网络传输十分顺畅，发送端总是能顺利收到确认，接收端宣告的接收窗口 $rwnd$ 始终为 64 KB。经过 4 个完整的 RTT（往返时间）后，发送端在第 5 个 RTT 期间向网络中注入的数据量是（ ）

- A. 8 KB
- B. 16 KB
- C. 17 KB
- D. 32 KB

【答案】：B

【解析】：

初始：第 1 个 RTT，发送前 $cwnd = 1$ KB，发送 1 KB，收到 ACK。此时发生倍增， $cwnd$ 变为 2 KB

第 2 个 RTT，发送前 $cwnd = 2$ KB，发送 2 KB，收到 ACK。倍增， $cwnd$ 变为 4 KB

第 3 个 RTT，发送前 $cwnd = 4$ KB，发送 4 KB，收到 ACK。倍增， $cwnd$ 变为 8 KB

第 4 个 RTT，发送前 $cwnd = 8$ KB，发送 8 KB，收到 ACK。倍增， $cwnd$ 变为 16 KB

第 5 个 RTT 开始发送时， $cwnd$ 就是 16 KB。因此它会在这个 RTT 内注入 16 KB 的数据。只有当这 16 KB 全部被确认后（第 5 个 RTT 结束）， $cwnd$ 才会依据拥塞避免算法增加到 17 KB

9. 【考点：TCP 拥塞控制】主机甲和主机乙进行大文件传输，当前拥塞窗口 $cwnd = 24$ 个 MSS，慢开始门限 $ssthresh = 16$ 个 MSS。此时，主机甲连续收到了 3 个对同一个旧数据包的重复确认（Duplicate ACKs）。若该连接严格执行 TCP Reno 版本的快重传与快恢复算法，且不考虑接收窗口限制，则当算法状态刚刚更新完毕，甲立即发送新的数据时，其拥塞窗口 $cwnd$ 和慢开始门限 $ssthresh$ 分别是（ ）

- A. $cwnd = 1$, $ssthresh = 12$
- B. $cwnd = 12$, $ssthresh = 12$
- C. $cwnd = 15$, $ssthresh = 12$
- D. $cwnd = 24$, $ssthresh = 12$

【答案】：B

【解析】：

当发送端收到 3 个冗余 ACK 时，意味着网络发生了轻微拥塞（个别丢包，非严重超时）

TCP 首先执行快重传：立即重传丢失的报文段

随后执行快恢复：

将慢开始门限减半： $ssthresh_{new} = cwnd_{old} / 2 = 24 / 2 = 12$

将拥塞窗口 $cwnd$ 的值设置为新的门限值： $cwnd_{new} = ssthresh_{new} = 12$

从此时起，直接进入拥塞避免阶段（线性增长）

10. 【考点：TCP/UDP 与 IP 分片】应用层生成了一个长度为 4000 字节的报文，要求传输给远端主机。网络路径中存在一条 MTU 为 1500 字节的以太网链路。假设采用 IPv4 协议（固定首部 20 字节）。对比使用 UDP（固定首部 8 字节）和使用 TCP（固定首部 20 字节）进行传输时的底层行为机制，下列说法正确的是（ ）

- A. 若使用 UDP，在发送主机上 UDP 层会将该报文切分为多个 UDP 数据报，以便 IP 层无需分片即可直接通过 MTU 限制
- B. 若使用 TCP，连接建立时的 MSS（最大报文段长度）协商机制通常会将其设为 1460 字节，从而保证生成的 IP 数据报刚好不需要分片
- C. 若使用 UDP，一旦 IP 层分片后的某一个碎片在网络中丢失，目的主机的 IP 层能够自动请求源主机的 IP 层仅重传丢失的那个碎片

D. 若使用 TCP，由于 TCP 能提供可靠传输，即使遇到极小 MTU 的链路导致 IP 层分片，TCP 也能依靠其内部序号对 IP 碎片进行精确确认

【答案】：B

【解析】：

A 错：UDP 协议不具备分段功能。它会将完整的 4000 字节连同 8 字节首部整体下交给 IP 层，由 IP 层负责切分为多个 IP 分片

B 正确：TCP 为了避免低效的 IP 分片，在握手阶段会协商 MSS (Maximum Segment Size)。在 1500 字节 MTU 链路下，MSS 推荐值 = MTU - IP 首部 - TCP 首部 = 1500 - 20 - 20 = 1460 字节。

TCP 主动将应用层数据切成 1460 字节的块，下层 IP 就不用再分片了

C 错：IP 层是不可靠的无连接服务，绝对不会进行重传。如果 UDP 的某个 IP 碎片丢失，整个 UDP 报文都会在接收端由于重组超时而遭到丢弃。UDP 也不会重传，只能依靠应用层超时重发全部 4000 字节

D 错：TCP 的确认机制是基于 TCP 首部的序号，该序号是面向传输层字节流的。一旦 TCP 段被下层 IP 强制分片，那些没有携带 TCP 首部的 IP 碎片对 TCP 来说是“透明”且不可见的，TCP 根本无法单独确认或重传某一个 IP 碎片。这也是 TCP 极力避免 IP 分片的原因

11. 【考点：UDP 校验】在开发分布式一致性协议（如 Raft）时，为了追求极致的选举速度，工程师决定将节点间的 Leader 心跳包从 TCP 改为基于 UDP 传输的轻量级 RPC。节点 A (IP: 192.168.1.10) 向节点 B (IP: 192.168.1.20) 发送了一个 UDP 心跳报文。关于该 UDP 数据报的校验和 (Checksum) 机制，以下说法完全正确的是 ()

- A. UDP 校验和只校验 UDP 首部 (8 字节) 和 UDP 数据部分，不关心网络层的 IP 地址信息
- B. 如果该心跳包在传输过程中经过了 NAT (网络地址转换) 路由器，其源 IP 发生改变，但由于 UDP 校验和不包含 IP 信息，到达节点 B 时校验依然会通过
- C. 计算 UDP 校验和时，会在 UDP 数据报之前临时添加一个 12 字节的“伪首部 (Pseudo-header)”。这个伪首部包含了源 IP 和目的 IP，但它既不下传给网络层，也不在网络上真实传输
- D. 若节点 B 收到该心跳包后，经计算发现 UDP 校验和错误，它会立刻丢弃该包，并通过 ICMP 协议向节点 A 发送“数据报错误”的重传请求

【答案】：C

【解析】：

A 选项错误：UDP 校验和不仅校验首部和数据，还必须校验伪首部

C 选项正确：伪首部 (12 字节) 包含了源 IP、目的 IP、协议号 (17) 和 UDP 长度。它的唯一作用就是为了计算校验和，从而确保这个 UDP 包没有被网络层投递到错误的 IP 或协议去。它存在于内存中，绝不会被真实发送到物理链路上

B 选项错误：正因为伪首部包含了 IP 地址，一旦 NAT 路由器修改了 IP，原来的 UDP 校验和就失效了。因此，NAT 路由器在修改 IP 的同时，必须重新计算并修改 UDP 的校验和，否则到达 B 时必定校验失败

D 选项错误：UDP 是不可靠传输。如果校验和出错，UDP 会静默丢弃该报文，绝对不会要求重传，通常也不会发送 ICMP 错误报告

12. 【考点：TCP 寻址与端口】基于参数服务器 (Parameter Server, PS) 架构的分布式训练网络所在的中心节点物理机启动了一个 TCP 监听服务，绑定在 0.0.0.0:8080 端口。随后，集群中的 100 个 Worker 节点同时向该 PS 发起连接请求，并成功建立 100 条处于 ESTABLISHED 状态的 TCP 连接

关于该中心节点操作系统的传输层状态，以下分析正确的是 ()

- A. 为了同时维持这 100 条独立的连接，PS 操作系统的网络栈除了监听端口 8080 外，还必须额外随机分配 100 个不同的本地端口 (如 50001~50100) 来处理数据
- B. 这 100 条连接在 PS 端全部复用同一个本地端口 8080。操作系统通过提取 TCP 段首部中的“源端口号”和“目的端口号”这个 2 元组，就能唯一区分是哪个 Worker 发来的数据
- C. 这 100 条连接在 PS 端全部复用同一个本地端口 8080。操作系统依靠一个完整的 4 元组 (源 IP, 源端口, 目的 IP, 目的端口) 来唯一确定一条 TCP 连接，并进行数据解复用
- D. 如果此时第 101 个 Worker 试图连接 8080 端口，由于 8080 端口已经被之前的 100 条连接“占用”，该连接请求 (SYN) 将被操作系统拒绝

【答案】：C

【解析】：

A 选项错误：服务器在建立连接后，其本地（目的）端口依然是 8080，绝对不会为每个客户端新开了一个本地端口。（否则 65535 个端口上限将导致服务器并发量惨不忍睹）

D 选项错误：监听套接字（Listening Socket）可以不断接受新的连接请求，只要系统的文件描述符（FD）和内存足够，8080 端口可以承载百万级并发连接

B 选项错误：仅仅依靠“源端口 + 目的端口”是无法唯一区分的。如果两台不同的 Worker 机器恰好用了相同的源端口（比如都是 54321）去连 PS 的 8080，2 元组就冲突了

C 选项正确：TCP 连接的灵魂标识是 4 元组（在严格上下文中加上协议号就是 5 元组）。无论多少个客户端连进来，在 PS 端看到的目的是（自己 IP，8080），由于每个 Worker 的（源 IP，源端口）组合必然不同，系统底层的哈希表能瞬间将到达的报文投递给对应的 Session 缓冲区（即解复用 Demultiplexing）

13. 【考点：TCP可靠传输】在一个基于 Kafka 的分布式日志收集系统中，生产者端底层通过 TCP 向 Broker 连续发送了 3 个报文段以保证日志的不丢失

这 3 个报文段携带的数据字节序号分别是：[501~1000]、[1001~1500]、[1501~2000]。由于数据中心交换机的负载均衡策略，这三个报文到达 Broker 的顺序发生了错乱，实际到达顺序为：第一段 [501~1000] 先到达，第三段 [1501~2000] 第二个到达，最后才是第二段

[1001~1500]

根据 TCP 的累计确认机制，Broker 在依次接收到这 3 个报文段后，发回给生产者的确认号（ACK）分别是（）

- A. 1001, 2001, 1501
- B. 1001, 1501, 2001
- C. 1001, 1001, 2001
- D. 1000, 1000, 2000

【答案】：C

【解析】：

确认号的定义：接收端期望收到的下一个字节的序号

接收动作 1：收到 [501~1000]。数据连续，期望下一个是 1001。发送 ACK = 1001

接收动作 2：收到 [1501~2000]。发生乱序，中间缺失了 [1001~1500]。TCP 必须发送确认号，但只能确认按序到达的最高字节。因此依然期待 1001。发送重复的 ACK = 1001

接收动作 3：收到迟到的 [1001~1500]。此时空缺被填补，从 501 到 2000 的数据在接收缓存中全部连续。期望下一个变为 2001。发送 ACK = 2001

最终序列为：1001, 1001, 2001。选 C

14. 【考点：UDP协议】在某大型多人在线战术竞技（MOBA）游戏中，为了降低玩家操作的网络延迟，客户端与服务器之间的英雄位置同步、技能释放指令等高频状态数据均使用 UDP 协议进行传输。关于该游戏架构中 UDP 协议的特性及首部结构，以下说法错误的是（）

- A. 即使游戏数据非常关键，UDP 在传输层依然不提供重传机制，如果有丢包，必须由游戏应用层（如状态插值或应用层重传）来处理
- B. UDP 首部只有 8 个字节，其包含的字段为：源端口、目的端口、长度和校验和
- C. 当游戏服务器收到 UDP 数据报并计算校验和发现错误时，它会自动向客户端回送一个 ICMP 差错报告报文，要求客户端重发
- D. 在计算 UDP 校验和时，会在 UDP 数据报前临时添加一个 12 字节的“伪首部”，该伪首部包含了源 IP 和目的 IP 地址，用于验证数据报是否被错误地路由到了其他主机

【答案】：C

【解析】：

A 正确：UDP 是尽力而为的传输，不保证可靠性。现代游戏通常在 UDP 之上自建可靠性协议（如 KCP 或 QUIC 的变种）

B 正确：UDP 首部极其简单，固定 8 字节

D 正确：这是 UDP 校验和的核心机制。伪首部只是临时参与计算，不真正在网上传输，目的是让

UDP 层也能验证一下 IP 地址是否匹配

C 错误：UDP 是无状态、无连接的。如果校验和出错，UDP 的唯一处理方式就是直接丢弃该数据报，绝对不会主动要求重发，也不会自动生成 ICMP 报文去纠正。应用层根本不会知道这个包曾经来过

15. 【考点：传输层寻址与端口】某互联网公司的网关入口部署了一台 LVS (Linux Virtual Server) 作为四层负载均衡器。该负载均衡器的公网虚拟 IP 为 114.114.114.114，它将外部所有访问 TCP 80 端口的流量透明地转发给后端的 50 台真实 Web 服务器

假设在某一时刻，有 20 万个不同 IP 地址的用户正在通过该负载均衡器与后端的 Web 集群保持活跃的 TCP 长连接。关于这些连接的识别与复用机制，以下推论正确的是 ()

- A. 由于 Web 服务只监听 80 端口，且 TCP 端口号只有 16 位 (最大 65535)，因此该系统理论上绝对不可能同时维持 20 万个连接
B. 传输层的解复用依赖于“四元组”。由于这 20 万个用户的源 IP 或源端口各不相同，即便目的 IP 和目的端口始终是 114.114.114.114:80，操作系统也能准确区分这 20 万个独立的 TCP 字节流
C. 后端 Web 服务器的 80 端口只负责监听新的连接请求，一旦连接建立，服务器会为每个用户分配一个新的随机端口 (如 40001、40002) 来进行数据传输，从而突破端口数量限制
D. TCP 首部的长度字段可以用来区分不同的连接，当端口号不够用时，底层网络会自动启用 IP 层的分片机制来复用物理链路

【答案】：B

【解析】：

A 错误：客户端受限于可用端口 (单机最多向同一目标发起 65535 个连接)，但服务器端的并发连接数不受端口数量限制，只受限于系统的内存和文件描述符数量

B 正确：在传输层，标识一个 TCP 已建立连接 (ESTABLISHED) 的主键是 (源 IP，源端口，目的 IP，目的端口)。对于服务器而言，目的 IP 和 80 端口是固定的，只要客户端的 IP 或端口有一个不同，四元组就是唯一的，就能被底层的 Socket 准确分发

C 错误：服务器 `accept()` 返回的是一个新的已连接套接字 (Socket FD)，但这个新套接字绑定的本地端口依然是 80 端口，并不会分配新的随机端口去进行数据传输

D 错误：TCP 首部没有用于区分连接的长度字段，区分连接完全靠四元组。IP 分片与连接复用毫无关系

16. 16. C

UDP 检验和的计算方法是二进制反码求和再取反，二进制反码求和的运算规则：① 从低位到高位逐列进行加法运算，0 和 0 相加得 0，0 和 1 相加得 1，1 和 1 相加得 0 并产生进位 1；② 若最高位相加后产生进位，则将该进位加到结果的最低位，该过程也称回卷。计算过程如下：

$$\begin{array}{r} 1011\ 1001\ 1011\ 0110 \\ +\ 0110\ 0101\ 1100\ 0101 \\ \hline [1]0001\ 1111\ 0111\ 1011 \longrightarrow \text{最高位产生了进位，最低位要加1} \\ +\ 1 \\ \hline 0001\ 1111\ 0111\ 1100 \longrightarrow \text{最后一步要取反} \\ 1110\ 0000\ 1000\ 0011 \end{array}$$

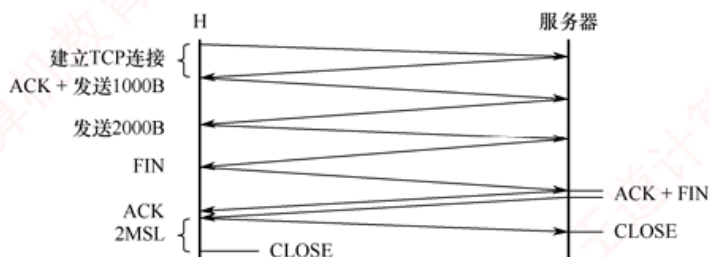
17. 17. B

NAT 在转发内网主机 H 的 UDP 报文时，会修改 IP 数据报的源 IP 地址和 UDP 首部的源端口号 (I 被修改)，而目的端口号 (II) 保持不变。UDP 首部中的总长度 (III) 仅由 UDP 数据自身决定，未发生变化。但由于 UDP 检验和 (IV) 的计算包含伪首部 (含源 IP 地址)，当源 IP 被 NAT 替换后，必须重新计算检验和，因此 IV 被修改。综上，仅 I 和 IV 被修改。

- 18.

59. D

TCP 连接建立的前两次握手耗时 1RTT。第三次握手段可携带 1000B 数据（或与数据段合并发送）。H 收到确认后，拥塞窗口增长至 2000B。因此第 3 个 RTT 内，可发送剩余 2000B 数据，至此数据传输完毕。第 4 个 RTT 开始时，H 向服务器发送 FIN 请求断开连接。题目问的是最少时间，因此服务器收到 FIN 后不再发送数据，并立即回复 ACK + FIN。第 4 个 RTT 结束时，H 收到服务器的 FIN，等待 2MSL 时间（60s）后进入 CLOSED 状态，总时间为 $40\text{ms} + 60\text{s} = 60.04\text{s}$ 。



19. 60. A

甲收到确认段（ack_seq = 3001），表明序号 2001~3000 的 1000 B 数据已接收。由于当前拥塞窗口小于阈值，处于慢启动阶段，每收到一个 ACK，拥塞窗口加 1 MSS，故由 2000 B 增至 3000B。接收窗口为 4000B。因此发送窗口 = $\min\{3000, 4000\} = 3000\text{B}$ 。此时甲已发送但尚未确认的数据为序号 3001~4000（1000 B），故可用窗口为 $3000 - 1000 = 2000\text{B}$ ，还可发送 2 个 TCP 段。

20. 61. B

UDP 是无连接的，客户发送请求后服务器直接返回应答，整个过程需约 1RTT，因此 UDP 的最少时间为 8ms。TCP 需先通过三次握手建立连接：客户发送 SYN，服务器回复 SYN-ACK，客户再发送 ACK（可携带应用请求）。从客户发送 SYN 起，到服务器收到请求共经历 1.5RTT，服务器响应后，再经 0.5RTT 返回客户，总计 2RTT。因此 TCP 的最少时间为 16ms。

21. 13. 【解答】

1) 由图 1 看出，源 IP 地址为 IP 分组头的第 13~16 个字节。在表 1 中，1、3、4 号分组的源 IP 地址均为 192.168.0.8 (c0a80008H)，所以 1、3、4 号分组是由 H 发送的。再观察 1、3、4 号分组的标识字段，分别是 9b, 9c, 9d，标识字段是一个计数器，每产生一个数据报就加 1，这也说明主机 H 先后发送了 1、3、4 号分组。

在表 1 中，1 号分组封装的 TCP 段的 SYN=1，ACK=0，seq=846b 41c5H；2 号分组封装的 TCP 段的 SYN=1，ACK=1，seq=e059 9fefH，ack=846b 41c6H；3 号分组封装的 TCP 段的 ACK=1，seq=846b 41c6H，ack=e059 9ff0H，所以 1、2、3 号分组完成了 TCP 连接的建立过程。

由于快速以太网数据帧有效载荷的最小长度为 46B，表 1 中 3、5 号分组的总长度为 40 (28H) 字节，小于 46B，其余分组总长度均大于 46B。所以 3、5 号分组通过快速以太网传输时需要填充。

2) 由 3 号分组封装的 TCP 段可知，发送应用层数据初始序号为 seq=846b 41c6H，由 5 号分组封装的 TCP 段可知，ack 为 seq=846b 41d6H，所以 S 已经收到的应用层数据的字节数为 $846b 41d6H - 846b 41c6H = 10H = 16\text{B}$ 。

3) 因为 S 发出的 IP 分组的标识=6811H，所以该分组所对应的是表 1 中的 5 号分组。S 发出的 IP 分组的 TTL=40H=64，5 号分组的 TTL=31H=49， $64 - 49 = 15$ ，所以可以推断该 IP 分组到达 H 时经过了 15 个路由器。

22.

14. 【解答】

- 1) 第二次握手报文段的 $\text{SYN}=1$, $\text{ACK}=1$ 。确认序号是第一次握手报文段的序号 $+1=101$ 。
- 2) 因为 S 的 TCP 接收缓存仅有数据存入而无数据取出, 当 S 收到第 8 个段时接收缓存还剩 12KB, 所以 H3 收到的第 8 个确认段所通告的接收窗口是 12KB。在慢开始阶段, 每收到一个对新报文段的确认, 拥塞窗口就加 1, 当收到对第 8 个报文段的确认后, H3 的拥塞窗口变为 9KB。H3 的发送窗口取接收窗口和拥塞窗口的最小值, 即 9KB。
- 3) H3 的发送窗口等于 0 时, 下一个待发送段的序号是 $20K+101=20\times 1024+101=20581$ 。H3 从发送第 1 个段到发送窗口等于 0 时刻为止, 共耗费了 5 个 RTT (每个 RTT 传输的数据量依次为 1KB、2KB、4KB、8KB、5KB), 平均数据传输速率是 $20KB\div(5\times 200\text{ms})=20\times 1024\times 8\text{b/s}=163.84\text{kb/s}$ 。

注 意

K 表示文件大小或描述存储空间时等于 1024, 这里通常用大写的 K; k 表示传输速率或描述网络通信时等于 1000, 这里通常用小写的 k。注意区分和转换。

- 4) t 时刻 H3 请求断开连接, 发出连接释放 FIN 报文段; S 收到后, 最短时间的情况是 S 已没有要发送的数据, 所以同时发出确认 ACK 报文段和连接释放 FIN 报文段, 即 S 直接跳过 CLOSE-WAIT 状态; H3 收到 FIN 报文段后必须发出确认, S 收到确认后进入 CLOSED 状态, 共经历 1.5 个 RTT, 因此 S 释放该连接的最短时间是 $1.5\times 200\text{ms}=300\text{ms}$ 。

23. 【考点: TCP段】主机 A 与主机 B 建立了一条 TCP 连接。已知发送路径上的瓶颈链路是一段采用 PPPoE 协议的宽带接入网, 其网络层的最大传输单元 (MTU) 限制为 1492 字节。为了完全避免 IP 层分片, TCP 在三次握手期间协商了最大报文段长度 (MSS)。假设使用标准的 IPv4 首部 (20 字节) 和 TCP 首部 (20 字节), 且接收端 B 所在的局域网是一个标准的 100Base-T 以太网 (MTU 为 1500 字节)。当 B 的网卡接收到 A 发送的一个达到最大 MSS 长度的数据段时, 其数据链路层完整的以太网 MAC 帧 (包含 14 字节 MAC 首部和 4 字节 FCS 尾部) 的总长度是 ()

- A. 1492 字节
- B. 1510 字节
- C. 1514 字节
- D. 1518 字节

【答案】: B

【解析】:

确定 MSS: 为了避免在 $\text{MTU}=1492$ 的瓶颈链路发生分片, MSS 的取值必须是: 瓶颈 $\text{MTU}-\text{IP 首部}-\text{TCP 首部}=1492-20-20=1452$ 字节

接收端 B 的网络层还原 IP 数据报: $\text{IP 数据报总长度}=1452(\text{TCP 载荷})+20(\text{TCP 首部})+20(\text{IP 首部})=1492$ 字节

接收端 B 的数据链路层封装: $\text{以太网帧总长度}=1492(\text{IP 数据报})+14(\text{MAC 首部})+4(\text{FCS})=1510$ 字节。由于 1510 未超过以太网的帧长 (1518 字节), 故不需要分片, 最终长度为 1510 字节

24. 【考点: UDP校验】某局域网内部主机 (IP: 192.168.1.100) 通过一个 NAT (网络地址端口转换) 路由器向外网 DNS 服务器 (IP: 8.8.8.8) 发送了一个 UDP 查询数据报。该 UDP 报文在穿过 NAT 路由器发往外网的过程中, 关于首部校验和 (Checksum) 的变化, 下列说法正确的是 ()
- A. 仅网络层的 IP 首部校验和必须重新计算, 因为源 IP 地址发生了改变
 - B. 仅传输层的 UDP 校验和必须重新计算, 因为源端口号发生了改变
 - C. IP 首部校验和与 UDP 校验和都必须由 NAT 路由器重新计算并修改
 - D. UDP 协议是无连接不可靠的, 其校验和是可选的, 因此 NAT 路由器通常会将其置为 0 以加速转发

【答案】: C

【解析】:

NAT 路由器会修改内网主机的“源 IP 地址”和“源端口号”

因为“源 IP 地址”属于 IP 首部, 所以 IP 首部的校验和必定改变, 需要重新计算

关键点：UDP 校验和的计算包含了一个“伪首部”。伪首部中包含了源 IP 地址和目的 IP 地址。由于 NAT 修改了源 IP 地址，同时也修改了 UDP 首部中的源端口号，这两者都会导致原本的 UDP 校验和失效。因此，NAT 路由器必须深入到传输层，重新计算并更新 UDP 的校验和

25. 【考点：TCP流量控制】主机 A 和主机 B 通过一条专线连接，该链路的物理层带宽为 100 Mbps（即 10^8 bps），单向传播时延为 50 ms。双方采用 TCP 协议进行大文件传输。假设忽略所有首部开销、处理时延和排队时延，且接收端处理极快（不考虑应用层读取瓶颈）。为了在稳定传输状态下让该链路的信道利用率达到 100%，主机 B 在 TCP 首部中通告的接收窗口（rwnd）最小值至少应为（ ）。（注：1 Byte = 8 bits）

- A. 625 KB
- B. 1.25 MB
- C. 2.5 MB
- D. 12.5 MB

【答案】：B

【解析】：

往返时延（RTT）= 单向传播时延 $\times 2 = 50 \text{ ms} \times 2 = 100 \text{ ms} = 0.1 \text{ s}$

要让信道利用率达到 100%，发送窗口必须等于或大于带宽时延乘积（BDP），这样才能保证在等待第一个 ACK 返回的时间内，链路上始终有数据在传输

带宽时延乘积（BDP）= 带宽 $\times$ RTT = $10^8 \text{ bps} \times 0.1 \text{ s} = 10^7 \text{ bits}$

将 bits 转换为 Bytes： $10^7 / 8 = 1,250,000 \text{ Bytes} = 1.25 \text{ MB}$

由于发送窗口受限于接收窗口（rwnd），所以接收窗口至少需要通告 1.25 MB

26. 【考点：TCP拥塞控制】某 TCP 发送端当前正处于“拥塞避免”阶段，其拥塞窗口（cwnd）为 24 个 MSS，慢开始门限（sssthresh）为 16 个 MSS。此时，底层 OSPF 网络中出现短暂的路由环路，导致该 TCP 连接连续发送的 5 个 IP 数据报在网络层循环转发，直到其 IP 首部中的 TTL（生存时间）减为 0 而被路由器全部丢弃。在此期间，发送端没有收到任何来自接收端的确认报文。当 TCP 发送端察觉到异常并刚刚完成状态更新时，其 cwnd 和 sssthresh 的值分别是（ ）

- A. cwnd = 12, sssthresh = 12
- B. cwnd = 1, sssthresh = 12
- C. cwnd = 16, sssthresh = 12
- D. cwnd = 1, sssthresh = 8

【答案】：B

【解析】：

路由环路导致报文因 TTL=0 被丢弃，这些报文永远无法到达接收端

既然接收端没有收到这批报文后的任何失序报文，就不可能产生 3 个重复的 ACK（冗余 ACK）

因此，发送端不会触发“快重传/快恢复”机制（选项A是快恢复的结果），而是会一直等待，直到重传计时器超时（RTO）

发生超时（Timeout）时，TCP 的标准动作是：将 sssthresh 更新为当前 cwnd 的一半（ $24 / 2 = 12$ ），然后将 cwnd 骤降为 1 个 MSS，并重新进入慢开始阶段

27. 【考点：TCP连接管理】某高并发的反向代理服务器（如 Nginx）负责接收客户端请求，并在内部主动向后端的应用服务器发起短连接（每次请求建立新的 TCP 连接，响应结束后立即关闭）。在业务高峰期，系统日志频繁报错“无法分配请求的地址（Cannot assign requested address）”，导致无法向后端建立新连接。网络管理员使用 netstat 命令查看，发现代理服务器上存在数万个处于同一种状态的 TCP 连接。根据 TCP 连接管理机制，这种状态最可能是（ ），且按照标准，该状态下的连接会持续维持（ ）时间

- A. CLOSE_WAIT 状态；维持时间由操作系统任意决定
- B. TIME_WAIT 状态；维持时间为 2MSL（最长报文段寿命的两倍）
- C. FIN_WAIT_2 状态；维持时间为 1MSL
- D. SYN_SENT 状态；维持直到收到后端的 SYN+ACK

【答案】：B

【解析】：

代理服务器“主动”向后端发起连接，且响应结束后“主动”关闭连接

在 TCP 四次挥手中，主动关闭连接的一方在发送出最后一个 ACK 后，必须进入 TIME_WAIT 状态，以确保网络中残余的数据包消失，并保证对方能收到最后一个 ACK

TIME_WAIT 状态的标准维持时间是 2MSL

由于短连接极多且频繁主动关闭，大量的本地临时端口（Ephemeral Ports）被 TIME_WAIT 状态占用无法释放，最终导致本地可用端口耗尽，无法再发起新的连接

28. 【考点：传输层提供的服务】主机 A 接收到一个来自网络的标准以太网数据帧。网卡硬件核对目的 MAC 地址无误后将其接收，并层层向上解封装。数据链路层发现帧类型字段为 0x0800，将其上交给网络层；网络层 IP 模块检查目的 IP 正确，且协议字段为 17，于是将其载荷上交给 UDP 模块。然而，UDP 模块在进行分用时发现，报文首部指示的目的端口号当前没有任何应用进程在监听。按照 TCP/IP 协议栈的标准规范，主机 A 接下来会采取的动作是（ ）

- A. 传输层丢弃该 UDP 数据报，并直接构造一个 TCP RST（复位）报文发送给源主机
- B. 网络层静默丢弃该 IP 数据报，不进行任何回复，以防止端口扫描攻击
- C. 传输层丢弃该报文，并通知网络层构造一个 ICMP “目的不可达（端口不可达）”差错报告报文发送回源主机
- D. 数据链路层构造一个源 MAC 不匹配的拒绝接收帧，通知局域网内的交换机

【答案】：C

【解析】：

MAC 地址匹配、IP 地址匹配，说明数据确实是发给本机的

IP 首部协议号 17 对应 UDP（若是 6 则对应 TCP）

当 UDP 报文到达传输层，但对应的目的端口没有进程监听时，UDP 会丢弃该报文

关键动作：UDP 会调用网络层的 ICMP 机制，向源 IP 触发一个 ICMP 目的不可达报文（具体代码为：端口不可达，Type 3, Code 3）。这也是很多网络扫描工具（如 nmap 的 UDP 扫描）探测对方端口是否开放的底层原理。（注：TCP 在同样情况下会回复 RST 报文，但题目明确说明这是 UDP 报文，故 A 错误）

29. （1）发送窗口大小与拥塞状态判断

过程推导：

第 1 轮次：cwnd = 1，发送后按时收到确认，cwnd 变为 2

第 2 轮次：cwnd = 2，发送后按时收到确认，cwnd 变为 4

第 3 轮次：cwnd = 4，发送后按时收到确认，cwnd 变为 8

第 4 轮次：此时 cwnd = 8 MSS。主机 A 的实际发送窗口 $W = \min(\text{cwnd}, \text{rwnd})$ 。由于 rwnd = 24，所以发送窗口 $W = \min(8, 24) = 8 \text{ MSS}$ ，即 8 KB

第 4 轮次结束后：cwnd 翻倍变为 16 MSS。此时刚好达到门限值 ssthresh = 16

第 5 轮次：由于 cwnd = ssthresh = 16，根据 TCP 协议规定，此时既可以继续使用慢开始，也可以转入拥塞避免（通常做法是转入拥塞避免）。因此进入拥塞避免阶段

发送窗口大小为 8 KB。第 5 个传输轮次处于拥塞避免阶段

（2）失序到达时的 ACK 确认原则

TCP 采用累计确认（Cumulative Acknowledgment）机制。当报文段失序到达时（例如第 2 个丢失，第 3、4、5 个到达），接收方只能确认按序到达的最后一个字节

主机 B 必须依据累计确认原则，在后续回复的 ACK 报文中，其确认号（ack）始终填写主机 B 期望收到的下一个按序字节的序号（即丢失的第 2 个报文段的第一个字节的序号）

（3）快重传与快恢复参数计算

过程推导：

在第 5 轮次中，cwnd = 16，成功发送并确认后，cwnd = 17（拥塞避免阶段，加法增大）

在第 6 轮次中，cwnd = 17。在此轮次发生报文丢失，触发 3 个冗余 ACK

执行状态更新：发生快重传时，新的门限值更新为当前拥塞窗口的一半： $\text{ssthresh} = \lfloor \text{cwnd} / 2 \rfloor = \lfloor 17 / 2 \rfloor = 8 \text{ MSS}$

按照标准 Reno 算法，进入快恢复阶段，将拥塞窗口设置为新的门限值加上 3（代表已经有 3 个包离开了网络并被缓冲）： $\text{cwnd} = \text{ssthresh} + 3 = 8 + 3 = 11 \text{ MSS}$

`ssthresh` 变为 8 MSS，`cwnd` 变为 11 MSS

(4) 初始序号与数据段序号

三次握手阶段，SYN 报文消耗一个序号（即使不携带数据）。主机 A 发送 SYN 报文（ $\text{seq} = 5000$ ）。建立连接后，第一个携带应用数据的 TCP 报文段的数据载荷将从上一个消耗序号的下一位开始

第一个携带数据的 TCP 段的序号是 $\text{seq} = 5001$

30. (1) UDP 伪首部字段填值

伪首部不随数据报发送，仅用于在发送端和接收端计算校验和，以验证数据报是否送达了正确的目的端（IP + 端口）

源 IP 地址：客户端 C 的 IP，即 `192.168.1.100`

目的 IP 地址：DNS 服务器的 IP，即 `192.168.1.1`

协议号（Protocol）：网络层 IPv4 首部中指示 UDP 协议的固定编号，为 17

UDP 长度：UDP 首部长度（固定的 8 字节）+ 数据载荷长度（42 字节）= $8 + 42 = 50$ 字节

源 IP 地址为 192.168.1.100，目的 IP 地址为 192.168.1.1，协议号为 17，UDP 长度为 50

(2) UDP 校验和作用域与出错处理

作用范围：UDP 校验和覆盖了整个 UDP 数据报，包括：伪首部、UDP 首部和 UDP 数据载荷三部分

出错操作：如果校验和出错，UDP 协议栈会直接静默丢弃该数据报，既不会向上层应用交付，也不会向发送端发送任何差错确认报文（无连接不可靠协议的特性）

(3) TCP 三次握手字段值

第一次握手（C → S）：请求建立连接

`SYN = 1`，`ACK = 0`（因为是请求建立，确认号无效）

`seq = x = 1000`

`ack = 0`（或者无效空置）

第二次握手（S → C）：同意建立连接并确认第一次握手

`SYN = 1`，`ACK = 1`

`seq = y = 3000`

`ack = x + 1 = 1001`（确认收到 C 的 SYN，期望下一个序号）

第一次握手：`SYN = 1`，`ACK = 0`，`seq = 1000`，`ack = 0`（或无效）

第二次握手：`SYN = 1`，`ACK = 1`，`seq = 3000`，`ack = 1001`

(4) TCP 状态机与安全漏洞

服务器在收到 SYN 并回复 SYN+ACK 后，处于等待客户端最后确认的半连接状态

进入状态：服务器 S 进入 `SYN-RCVD`（同步收到）状态

典型攻击：这种半连接状态会占用服务器内核资源。若攻击者伪造大量不存在的源 IP 发送 SYN 请求且从不进行第三次握手，耗尽服务器的半连接队列，这就是典型的 SYN 洪泛攻击（SYN Flooding Attack）（或属于 DoS/DDoS 攻击的一种）